

Introdução a Algoritmos

Aula 11 – Matrizes

Raoni F. S. Teixeira · 2s/2026

Objetivos desta aula

Ao final desta aula, você deverá ser capaz de:

1. Declarar e acessar matrizes bidimensionais, relacionando os índices $[i][j]$ às dimensões linha e coluna;
2. Implementar os padrões de percurso de matriz com laços aninhados: soma total, soma por linha/coluna e diagonal principal;
3. Explicar o armazenamento row-major e suas implicações para a eficiência de percurso linha a linha versus coluna a coluna;
4. Passar matrizes para funções com a assinatura correta, justificando por que o número de colunas é obrigatório no parâmetro.

1 O que um vetor não consegue representar

Na Aula 9, vimos que um vetor organiza dados numa dimensão — uma sequência linear de elementos. Mas muitos problemas têm estrutura bidimensional: a tabela de notas de uma turma tem linhas (alunos) e colunas (provas); um tabuleiro de xadrez tem 8 linhas e 8 colunas; uma imagem digital é uma grade de pixels, cada um com um valor de intensidade.

Poderíamos tentar representar, por exemplo, as notas de 3 alunos em 4 provas com vetores separados:

```
float notas_aluno0[4] = {7.0, 8.5, 6.0, 9.0};  
float notas_aluno1[4] = {5.5, 7.0, 8.0, 6.5};  
float notas_aluno2[4] = {9.0, 9.5, 8.5, 10.0};
```

Isso funciona para 3 alunos, mas não escala — e não há como passar “a tabela inteira” para uma função de forma organizada. Uma **matriz** resolve o problema: ela generaliza o vetor para duas dimensões, organizando dados em linhas e colunas, acessíveis por dois índices.

Continua sendo a mesma peça “dados” da Aula 1: variável, vetor e matriz guardam a mesma coisa — valores do mesmo tipo — em complexidade crescente.

2 Declaração e acesso

Definição — Matriz

Uma matriz é um vetor bidimensional: um conjunto de elementos do mesmo tipo organizados em

linhas e colunas, acessados por dois índices — o primeiro para a linha, o segundo para a coluna. Em C, ambos os índices começam em zero.

A declaração especifica o tipo, o nome e as duas dimensões:

```
float notas[3][4]; /* 3 linhas, 4 colunas – 12 elementos no total */
int tabuleiro[8][8];
```

O acesso usa dois pares de colchetes:

```
notas[0][0] = 7.0; /* linha 0, coluna 0 */
notas[1][3] = 6.5; /* linha 1, coluna 3 */
notas[2][2] = 8.5; /* linha 2, coluna 2 */
```

A forma de ler `m[i][j]` é: “o elemento da linha *i*, coluna *j*”. Uma forma de lembrar: **i** de **row** (linha), **j** de **col** (coluna) — a ordem é sempre linha antes de coluna, assim como em matemática.

2.1 Inicialização

```
/* Lista aninhada – cada sublista inicializa uma linha */
int m[2][3] = {
    {1, 2, 3}, /* linha 0 */
    {4, 5, 6} /* linha 1 */
};

/* Zerar tudo */
int m[3][3] = {0};
```

3 Matrizes na memória

Assim como vetores, matrizes são armazenadas em blocos contíguos de memória. C usa a convenção **row-major**: os elementos são armazenados linha por linha — todos os elementos da linha 0 primeiro, depois todos da linha 1, e assim por diante.

Para `int m[2][3]`:

Índice	[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]
Endereço	1000	1004	1008	1012	1016	1020
Valor	1	2	3	4	5	6

Linha 0 (azul): endereços 1000–1008. Linha 1 (laranja): 1012–1020. As linhas são contíguas — o último elemento da linha 0 está imediatamente antes do primeiro elemento da linha 1.

Isso significa que `m[i][j]` está no endereço `base + (i * cols + j) * sizeof(tipo)`. Para `m[1][2]`: $1000 + (1 \cdot 3 + 2) \cdot 4 = 1000 + 20 = 1020$. ✓

Consequência prática: percorrer a matriz **linha por linha** (variando *j* no laço interno) acessa posições consecutivas de memória — o que é mais eficiente do que percorrer coluna por coluna. Para matrizes grandes, essa diferença é significativa.

4 As três perguntas aplicadas a matrizes

Matrizes exigem laços aninhados — um para as linhas, outro para as colunas. As três perguntas da Aula 5 se aplicam a cada laço:

Laço externo (linhas):

1. O que se repete? — processar uma linha inteira.
2. Quantas vezes? — `nlin` vezes (número de linhas).
3. O que muda? — o índice de linha `i`.

Laço interno (colunas):

1. O que se repete? — a operação sobre um elemento.
2. Quantas vezes? — `ncol` vezes (número de colunas).
3. O que muda? — o índice de coluna `j`.

O esqueleto padrão para percorrer toda a matriz:

```
for (int i = 0; i < nlin; i++) {           /* para cada linha */
    for (int j = 0; j < ncol; j++) {       /* para cada coluna */
        /* opera sobre m[i][j] */
    }
}
```

5 Padrões fundamentais

5.1 Leitura e impressão

```
/* Leitura */
for (int i = 0; i < nlin; i++)
    for (int j = 0; j < ncol; j++)
        scanf("%d", &m[i][j]);

/* Impressao formatada — cada linha numa linha de texto */
for (int i = 0; i < nlin; i++) {
    for (int j = 0; j < ncol; j++)
        printf("%4d", m[i][j]);
    printf("\n");           /* quebra de linha ao fim de cada linha */
}
```

O `printf("\n")` fora do laço interno é o detalhe que separa as linhas visualmente. Esquecê-lo imprime tudo numa única linha horizontal.

5.2 Soma total, por linha e por coluna

```
/* Soma total */
int total = 0;
for (int i = 0; i < nlin; i++)
    for (int j = 0; j < ncol; j++)
        total += m[i][j];

/* Soma de cada linha — resultado e um vetor de nlin valores */
```

```

for (int i = 0; i < nlin; i++) {
    int soma_linha = 0;
    for (int j = 0; j < ncol; j++)
        soma_linha += m[i][j];
    printf("Linha %d: %d\n", i, soma_linha);
}

/* Soma de cada coluna – resultado e um vetor de ncol valores */
for (int j = 0; j < ncol; j++) {
    int soma_col = 0;
    for (int i = 0; i < nlin; i++)
        soma_col += m[i][j];
    printf("Coluna %d: %d\n", j, soma_col);
}

```

Observe que na soma por coluna o laço externo percorre j e o interno percorre i — o oposto do padrão usual. Isso acessa elementos em colunas, saltando entre linhas na memória. Para matrizes pequenas não faz diferença; para matrizes grandes, é menos eficiente.

5.3 Diagonal principal

A diagonal principal de uma matriz quadrada contém os elementos onde $i == j$: $m[0][0]$, $m[1][1]$, $m[2][2]$, ...

```

/* Soma da diagonal principal (matriz quadrada n x n) */
int diag = 0;
for (int i = 0; i < n; i++)
    diag += m[i][i]; /* apenas um laço – i indexa linha e coluna */

```

5.4 Transposta

A transposta de uma matriz $m[nlin][ncol]$ é uma matriz $t[ncol][nlin]$ onde $t[j][i] = m[i][j]$ — linhas e colunas são trocadas:

```

int t[NCOL][NLIN];
for (int i = 0; i < nlin; i++)
    for (int j = 0; j < ncol; j++)
        t[j][i] = m[i][j];

```

6 Matrizes e funções

Passar uma matriz para uma função em C exige declarar o número de colunas no parâmetro — o compilador precisa dessa informação para calcular o endereço de $m[i][j]$ (lembre: $base + i*ncol*sizeof + j*sizeof$). O número de linhas pode ser omitido ou passado como parâmetro separado.

```

#define NCOL 4

void imprime(int m[][NCOL], int nlin) {
    for (int i = 0; i < nlin; i++) {
        for (int j = 0; j < NCOL; j++)
            printf("%4d", m[i][j]);
        printf("\n");
    }
}

```

A chamada passa o nome da matriz (sem colchetes) e o número de linhas:

```
int notas[3][NCOL] = {{7,8,6,9},{5,7,8,6},{9,9,8,10}};
imprime(notas, 3);
```

Atenção

O número de colunas **precisa** ser uma constante em tempo de compilação na assinatura da função. Isso é uma limitação de C que decorre do modelo de memória row-major: sem saber o número de colunas, o compilador não consegue gerar o código para `m[i][j]`. Matrizes de tamanho totalmente variável exigem alocação dinâmica — tema de aulas futuras.

7 Exemplo integrador: processamento de imagem

Uma imagem em tons de cinza pode ser representada como uma matriz de inteiros, onde cada elemento é a **intensidade** de um pixel — um valor entre 0 (preto) e 255 (branco). Uma imagem de largura *W* e altura *H* é uma matriz `int img[H][W]`.

Três operações básicas de processamento ilustram os padrões de acesso que aprendemos:

7.1 Ajuste de brilho

Somar uma constante a todos os pixels aumenta o brilho; subtrair, diminui. O resultado deve ser mantido no intervalo `[0, 255]`:

```
#define H 4
#define W 5

/*
 * Ajusta o brilho de img somando delta a cada pixel.
 * Valores sao limitados ao intervalo [0, 255].
 */
void ajusta_brilho(int img[][W], int h, int delta) {
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < W; j++) {
            img[i][j] += delta;
            if (img[i][j] > 255) img[i][j] = 255;
            if (img[i][j] < 0)   img[i][j] = 0;
        }
    }
}
```

Percurso simples — todos os elementos tratados da mesma forma. O índice `[i][j]` aparece apenas uma vez por iteração.

7.2 Espelhamento horizontal

Espelhar horizontalmente inverte a ordem das colunas em cada linha. A coluna *j* troca com a coluna `W-1-j`:

```
/*
 * Espelha img horizontalmente (espelho vertical — como olhar num espelho).
 * Opera no proprio buffer, sem matriz auxiliar.
 */
```

```

void espelha_horizontal(int img[][W], int h) {
    for (int i = 0; i < h; i++) {
        int esq = 0, dir = W - 1;
        while (esq < dir) {
            int temp = img[i][esq];
            img[i][esq] = img[i][dir];
            img[i][dir] = temp;
            esq++;
            dir--;
        }
    }
}

```

O laço externo percorre linhas; o interno usa dois índices opostos — o mesmo padrão de inversão de vetor da Aula 9, aplicado a cada linha da matriz.

7.3 Espelhamento vertical

Espelhar verticalmente inverte a ordem das linhas. A linha i troca com a linha $h-1-i$:

```

/*
 * Espelha img verticalmente (vira de cabeça para baixo).
 * Opera no proprio buffer, sem matriz auxiliar.
 */
void espelha_vertical(int img[][W], int h) {
    int cima = 0, baixo = h - 1;
    while (cima < baixo) {
        for (int j = 0; j < W; j++) {
            int temp = img[cima][j];
            img[cima][j] = img[baixo][j];
            img[baixo][j] = temp;
        }
        cima++;
        baixo--;
    }
}

```

Aqui o laço externo percorre pares de linhas a trocar; o laço interno troca elemento a elemento dentro de cada par.

7.4 Programa completo

```

#include <stdio.h>
#define H 4
#define W 5

void imprime(int img[][W], int h) {
    for (int i = 0; i < h; i++) {
        for (int j = 0; j < W; j++)
            printf("%4d", img[i][j]);
        printf("\n");
    }
}

/* ... definicoes de ajusta_brilho, espelha_horizontal, espelha_vertical ... */

```

```
int main() {
    int img[H][W] = {
        { 10, 20, 30, 40, 50},
        { 60, 70, 80, 90, 100},
        {110, 120, 130, 140, 150},
        {160, 170, 180, 190, 200}
    };

    printf("Original:\n");
    imprime(img, H);

    ajusta_brilho(img, H, 50);
    printf("\nApos brilho +50:\n");
    imprime(img, H);

    espelha_horizontal(img, H);
    printf("\nApos espelho horizontal:\n");
    imprime(img, H);

    espelha_vertical(img, H);
    printf("\nApos espelho vertical:\n");
    imprime(img, H);

    return 0;
}
```

Saída esperada:

Original:

```
10 20 30 40 50
60 70 80 90 100
110 120 130 140 150
160 170 180 190 200
```

Apos brilho +50:

```
60 70 80 90 100
110 120 130 140 150
160 170 180 190 200
210 220 230 240 250
```

Apos espelho horizontal:

```
100 90 80 70 60
150 140 130 120 110
200 190 180 170 160
250 240 230 220 210
```

Apos espelho vertical:

```
250 240 230 220 210
200 190 180 170 160
150 140 130 120 110
100 90 80 70 60
```

Observe como cada operação transforma a matriz de forma visualmente interpretável — um pixel com valor 210 após brilho +50 representa um pixel que estava em 160 e foi clareado. O processamento de imagem torna os padrões de acesso à matriz concretos e verificáveis.

8 Recapitulação: vetor vs. matriz

Aspecto	Vetor	Matriz
Dimensões	1 — índice único $v[i]$	2 — linha e coluna $m[i][j]$
Declaração	<code>int v[N]</code>	<code>int m[NLIN][NCOL]</code>
Percurso	Um laço	Dois laços aninhados
Memória	Bloco contíguo linear	Bloco contíguo row-major
Em funções	<code>int v[]</code> ou <code>int *v</code>	<code>int m[][NCOL]</code> — colunas obrigatórias
Inicialização	{1, 2, 3}	{{1,2},{3,4}}

9 Exercícios

- Escreva uma função `int soma_diagonal(int m[][N], int n)` que retorna a soma dos elementos da diagonal principal de uma matriz quadrada $n \times n$. Escreva também `int soma_diagonal_sec(int m[][N], int n)` para a diagonal secundária (de $m[0][n-1]$ até $m[n-1][0]$). Qual é a fórmula do índice de coluna para a diagonal secundária?
- Escreva uma função `int e_simetrica(int m[][N], int n)` que retorna 1 se a matriz quadrada $n \times n$ for simétrica ($m[i][j] == m[j][i]$ para todo i, j) e 0 caso contrário. Não percorra a matriz inteira — basta verificar os elementos acima da diagonal principal.
- Escreva uma função `void rotaciona90(int m[][N], int res[][N], int n)` que armazena em `res` a rotação de 90 graus no sentido horário de `m`. (Dica: o elemento $m[i][j]$ vai para $res[j][n-1-i]$.) Verifique com a matriz identidade e uma matriz 3×3 simples.
- Escreva um programa que leia uma matriz 3×3 de inteiros e determine se ela é um **quadrado mágico**: uma matriz onde a soma de cada linha, cada coluna e as duas diagonais são todas iguais. Teste com o quadrado mágico clássico:

```
2 7 6
9 5 1
4 3 8
```

(Todas as somas valem 15.)

- (Desafio) Escreva `void produto(int a[][N], int b[][N], int c[][N], int n)` que calcula o produto matricial $c = a * b$ para matrizes quadradas $n \times n$. O elemento $c[i][j]$ é a soma dos produtos $a[i][k] * b[k][j]$ para $k = 0, \dots, n-1$. Quantos laços são necessários? Quantas multiplicações para $n = 100$?